

Trabajo Práctico: Métodos Numéricos

Contenidos

Definición del Problema	1
(a) Punto mínimo de la soga	2
Resolviendo Newton-Raphson para encontrar γ	2
(b) Verificar la longitud de la soga	6
(c) Ajuste de polinomio	8
(d) Separación máxima	11

[Para una mejor visualización del trabajo](#)

Definición del Problema

Una soga de longitud L se encuentra atada a dos postes a una altura h , con sus centros separados entre sí una cierta distancia $2a$, como muestra la figura. Considerando un sistema de coordenadas cartesianas en el cual el eje x se sitúa paralelo al suelo y el eje y se encuentra exactamente a igual distancia de ambos postes, se puede describir cada punto de la soga mediante la siguiente ecuación:

$$y(x) = \frac{1}{\gamma} (\cosh(\gamma x) - \cosh(\gamma a)) + h$$

,

donde x , y , a están en metros, y γ es un parámetro cuyo valor es positivo y cumple la siguiente ecuación:

$$L = \frac{2}{\gamma} \sinh(\gamma a)$$

(a) Punto mínimo de la sogá

i Note

Si $L = 8m$, $a = 3m$ y $h = 2,5m$, grafique los puntos de la sogá y determine a qué altura se encuentra el punto más bajo de la misma.

Reemplazando los valores dados en la expresión de L se obtiene la siguiente ecuación para γ :

$$8 = \frac{2}{\gamma} \sinh(3\gamma)$$

De esta expresión no podemos simplemente *despejar* γ , sino que debemos resolverla numéricamente. Para esto, podemos reescribirla como una función igualada a cero y luego utilizar un método de búsqueda de raíces para encontrar el valor de γ que satisface la ecuación:

$$f(\gamma) = \sinh(3\gamma) - 4\gamma = 0$$

En este caso se utiliza el método de Newton-Raphson para encontrar la raíz de esta función, lo que nos dará el valor de γ que necesitamos para calcular la altura del punto más bajo de la sogá. Luego, con el valor de γ calculado, podemos usar la ecuación de $y(x)$ para encontrar la altura mínima en el punto $x = 0$. Finalmente, se generan los puntos necesarios para graficar la forma de la sogá y se muestra la gráfica resultante.

Resolviendo Newton-Raphson para encontrar γ

El método encuentra las raíces de una función iterativamente utilizando la fórmula:

$$\gamma_{n+1} = \gamma_n - \frac{f(\gamma_n)}{f'(\gamma_n)}$$

La derivada de $f(\gamma)$ es:

$$f'(\gamma) = 3 \cosh(3\gamma) - 4$$

```

import numpy as np

# Parámetros dados
L = 8.0
a = 3.0
h = 2.5

# Definición de la función y su derivada
def f(gamma):
    return np.sinh(3 * gamma) - 4 * gamma

def df(gamma):
    return 3 * np.cosh(3 * gamma) - 4

# Implementación del Método de Newton-Raphson
def newton_raphson(func, dfunc, x0, tol=1e-6, max_iter=20):
    x_n = x0
    print(f"{'Iteración':<10} | {'gamma_n':<15} | {'f(gamma_n)':<15}")
    print("-" * 45)

    for n in range(max_iter):
        f_val = func(x_n)

        # Criterio de parada
        if abs(f_val) < tol:
            print("-" * 45)
            print(f"Convergencia alcanzada en {n} iteraciones.")
            return x_n

        df_val = dfunc(x_n)

        # Por si la derivada llega a ser cero
        if df_val == 0:
            print("Error: La derivada es cero")
            return None

        print(f"{n:<10} | {x_n:<15.8f} | {f_val:<15.8e}")

        # iteración de Newton
        x_n = x_n - f_val / df_val

    print("Se alcanzó el número máximo de iteraciones sin converger.")

```

```

    return x_n

gamma_inicial = 0.5
gamma_calculado = newton_raphson(f, df, gamma_inicial)

print(f"\nValor final de gamma calculado: {gamma_calculado:.8f}")

```

Iteración	gamma_n	f(gamma_n)
0	0.50000000	1.29279455e-01
1	0.45771352	1.63554910e-02
2	0.45060725	4.16393909e-04

Convergencia alcanzada en 3 iteraciones.

Valor final de gamma calculado: 0.45041667

Como el problema de la sogá es simétrico respecto al eje y , el punto más bajo se encuentra en $x = 0$. Por lo tanto, para calcular la altura mínima de la sogá, simplemente evaluamos $y(0)$ usando el valor de γ que acabamos de calcular:

$$y_{min} = y(0) = \frac{1}{\gamma} (\cosh(0) - \cosh(\gamma a)) + h = \frac{1}{\gamma} (1 - \cosh(\gamma a)) + h$$

Reemplazando por $\gamma = 0,45041667$:

$$y_{min} = \frac{1}{0,45041667} (1 - \cosh(0,45041667 \cdot 3)) + 2,5$$

```

y_min = (1 / gamma_calculado) * (1 - np.cosh(gamma_calculado * a)) + h
print(f"Altura del punto más bajo: {y_min:.4f} m")

```

Altura del punto más bajo: 0.1453 m

```

import matplotlib.pyplot as plt

x = np.linspace(-a, a, 200)
y = (1 / gamma_calculado) * (np.cosh(gamma_calculado * x) -
    np.cosh(gamma_calculado * a)) + h

```

```

fig, ax = plt.subplots(figsize=(8, 5))

ax.plot(x, y, color='blue', linewidth=2.5, label='Soga')

ax.plot([-a, -a], [0, h], color='black', linestyle='--', linewidth=2)
ax.plot([a, a], [0, h], color='black', linestyle='--', linewidth=2)

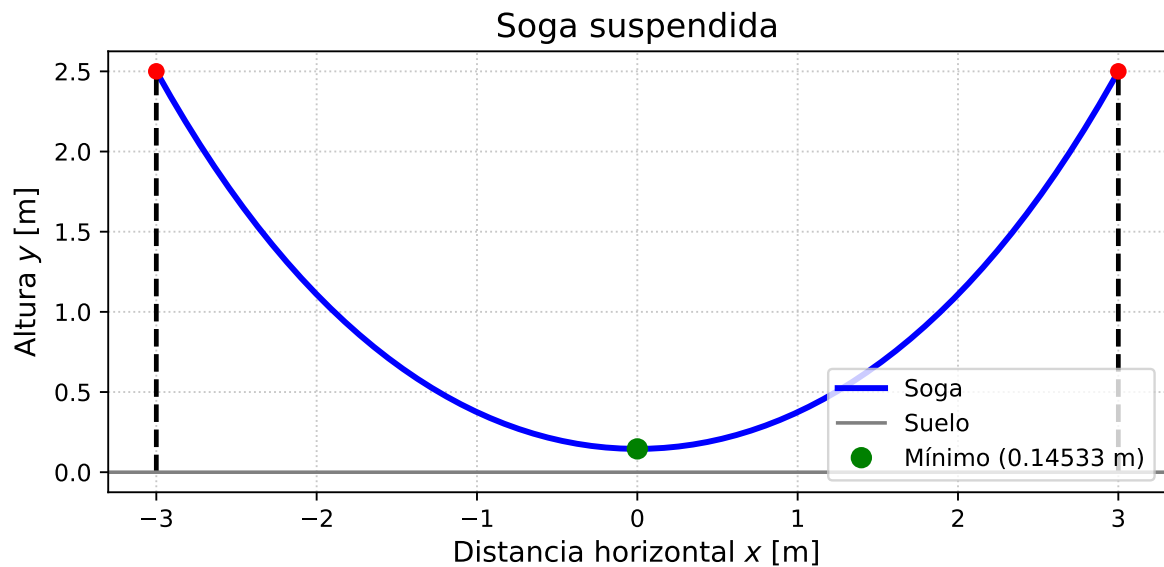
ax.axhline(0, color='gray', linewidth=1.5, label='Suelo')

ax.plot([-a, a], [h, h], 'ro', markersize=6)

ax.plot(0, y_min, 'go', markersize=8, label=f'Mínimo ({y_min:.5f} m)')

ax.set_title('Soga suspendida', fontsize=14)
ax.set_xlabel('Distancia horizontal  $x$  [m]', fontsize=12)
ax.set_ylabel('Altura  $y$  [m]', fontsize=12)
ax.grid(True, linestyle=':', alpha=0.7)
ax.legend(loc='lower right')
ax.set_aspect('equal')

```



(b) Verificar la longitud de la sogá

Note

Verifique que la longitud de la sogá es $L = 8m$.

Resolviendo de forma analítica la integral de la longitud de la curva dada por $y(x)$:

$$y'(x) = \frac{1}{\gamma} \gamma \sinh(\gamma x) = \sinh(\gamma x)$$

Luego:

$$L = \int_{-a}^a \sqrt{1 + (y'(x))^2} dx = \int_{-a}^a \sqrt{1 + \sinh^2(\gamma x)} dx$$

Por la identidad fundamental de las funciones hiperbólicas $\cosh^2(z) - \sinh^2(z) = 1$, se puede reescribir la raíz como:

$$L = \int_{-a}^a \sqrt{\cosh^2(\gamma x)} dx = \int_{-a}^a \cosh(\gamma x) dx$$

Resolviendo analíticamente:

$$L = \left[\frac{1}{\gamma} \sinh(\gamma x) \right]_{-a}^a = \frac{1}{\gamma} (\sinh(\gamma a) - \sinh(-\gamma a))$$

El seno hiperbólico es una función impar, por lo que $\sinh(-z) = -\sinh(z)$, lo que nos permite simplificar la expresión a:

$$L = \frac{2}{\gamma} \sinh(\gamma a)$$

Esta es la misma expresión que teníamos inicialmente.

Podemos realizar la integración numérica utilizando el Método de Simpson 1/3 para verificar que el resultado coincide con $L = 8m$. Para este método es necesario evaluar la función integrando en varios puntos dentro del intervalo de integración, y luego aplicar la fórmula de Simpson para obtener una aproximación de la integral:

$$L \approx \frac{h}{3} \left(f(x_0) + 4 \sum_{i=1,3,5,\dots}^{n-1} f(x_i) + 2 \sum_{i=2,4,6,\dots}^{n-2} f(x_i) + f(x_n) \right)$$

```

def integrando(x):
    return np.cosh(gamma_calculado * x)

# Implementación del Método de Simpson 1/3
def simpson_13(func, lim_inf, lim_sup, n_intervalos):
    if n_intervalos % 2 != 0:
        raise ValueError("El número de intervalos debe ser par.")

    h = (lim_sup - lim_inf) / n_intervalos
    x = np.linspace(lim_inf, lim_sup, n_intervalos + 1)
    y = func(x)

    # Aplicamos la fórmula: (h/3) * (y0 + 4*(y_impares) + 2*(y_pares) + yn)
    extremos = y[0] + y[-1]

    # Términos impares
    pares = 4 * np.sum(y[1:-1:2])

    # Términos pares
    impares = 2 * np.sum(y[2:-2:2])

    integral = (h / 3) * (extremos + pares + impares)
    return integral

n = 50

longitud_calculada = simpson_13(integrando, -a, a, n)

print(f"Longitud de la sog: {longitud_calculada:.8f} m")
print(f"Longitud teórica esperada: 8.00000000 m")
print(f"Error absoluto: {abs(8.0 - longitud_calculada):.2e} m")

```

```

Longitud de la sog: 8.00000169 m
Longitud teórica esperada: 8.00000000 m
Error absoluto: 1.69e-06 m

```

(c) Ajuste de polinomio

Note

Tomando 8 puntos igualmente espaciados sobre el eje x , considerando los extremos de la sogá, obtenga el polinomio de grado menor o igual a 4 que mejor ajusta esos puntos de la sogá en el sentido de cuadrados mínimos. Calcule el error que se comente con este ajuste al determinar el punto más bajo de la sogá.

El objetivo será encontrar un polinomio de como máximo grado 4 de la forma:

$$P_4(x) = c_0 + c_1x + c_2x^2 + c_3x^3 + c_4x^4$$

Tenemos un conjunto $m = 8$ de puntos (x_i, y_i) , por lo que tenemos 8 ecuaciones y 5 incógnitas ($c = [c_0, c_1, c_2, c_3, c_4]^T$). Esto nos da un sistema sobredeterminado, el cual se puede resolver utilizando el método de mínimos cuadrados. Para esto, se construye la matriz de diseño A y el vector de observaciones Y :

$$A = \begin{bmatrix} 1 & x_0 & x_0^2 & x_0^3 & x_0^4 \\ 1 & x_1 & x_1^2 & x_1^3 & x_1^4 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_7 & x_7^2 & x_7^3 & x_7^4 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_7 \end{bmatrix}$$

Para convertir el sistema sobredeterminado $Ac = y$ en un sistema compatible, se multiplica ambos lados por la transpuesta de A , lo que lo convierte en un sistema cuadrado de 5×5 :

$$(A^T A) c = A^T y$$

El sistema resultante se puede resolver utilizando métodos de álgebra lineal para encontrar los coeficientes del polinomio $P_4(x)$ que mejor ajusta los puntos de la sogá en el sentido de mínimos cuadrados. Luego, con el polinomio ajustado, se puede calcular la altura del punto más bajo de la sogá evaluando $P_4(0)$ y comparándola con el valor real obtenido anteriormente para determinar el error cometido por el ajuste.

En este caso se utiliza el método de `numpy.linalg.solve`.

```
def sogá(x):  
    return (1 / gamma_calculado) * (np.cosh(gamma_calculado * x) - np.cosh(gamma_calculado *  
  
x_puntos = np.linspace(-a, a, 8)
```



```

y_puntos = sog(x_puntos)

A = np.column_stack([x_puntos**0, x_puntos**1,
                     x_puntos**2, x_puntos**3,
                     x_puntos**4])

AtA = A.T @ A
Aty = A.T @ y_puntos

coeficientes = np.linalg.solve(AtA, Aty)
c0, c1, c2, c3, c4 = coeficientes

print("Coeficientes del polinomio P4(x):")
print(f"c0 = {c0: .6f}")
print(f"c1 = {c1: .6f}")
print(f"c2 = {c2: .6f}")
print(f"c3 = {c3: .6f}")
print(f"c4 = {c4: .6f}")

y_min_real = sog(0)
y_min_aprox = c0 # P4(0) = c0

error_minimo = abs(y_min_real - y_min_aprox)

print(f"\nAltura mínima real de la sog: {y_min_real:.6f} m")
print(f"Altura mínima aproximada P4(0): {y_min_aprox:.6f} m")
print(f"Error absoluto en la aproximación: {error_minimo:.6e} m")

x_continuo = np.linspace(-a, a, 200)
y_continuo = sog(x_continuo)

y_pol = c0 + c1*x_continuo + c2*(x_continuo**2) + c3*(x_continuo**3) + c4*(x_continuo**4)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x_continuo, y_continuo, 'b-', linewidth=2, label='Real', alpha=0.7)
ax.plot(x_continuo, y_pol, 'r--', linewidth=2, label='Ajuste', alpha=0.9)
ax.plot(x_puntos, y_puntos, 'ko', markersize=6, label='Puntos de ajuste')

# Marcamos el error en el centro
ax.plot(0, y_min_real, 'bx', markersize=10, label='Mínimo real')
ax.plot(0, y_min_aprox, 'gx', markersize=10, label='Mínimo aprox.')

```

```

ax.set_title('Ajuste por Cuadrados Mínimos (Grado 4)', fontsize=14)
ax.set_xlabel('Posición  $x$  [m]')
ax.set_ylabel('Altura  $y$  [m]')
ax.grid(True, linestyle=':')
ax.legend()

```

Coeficientes del polinomio $P_4(x)$:

```

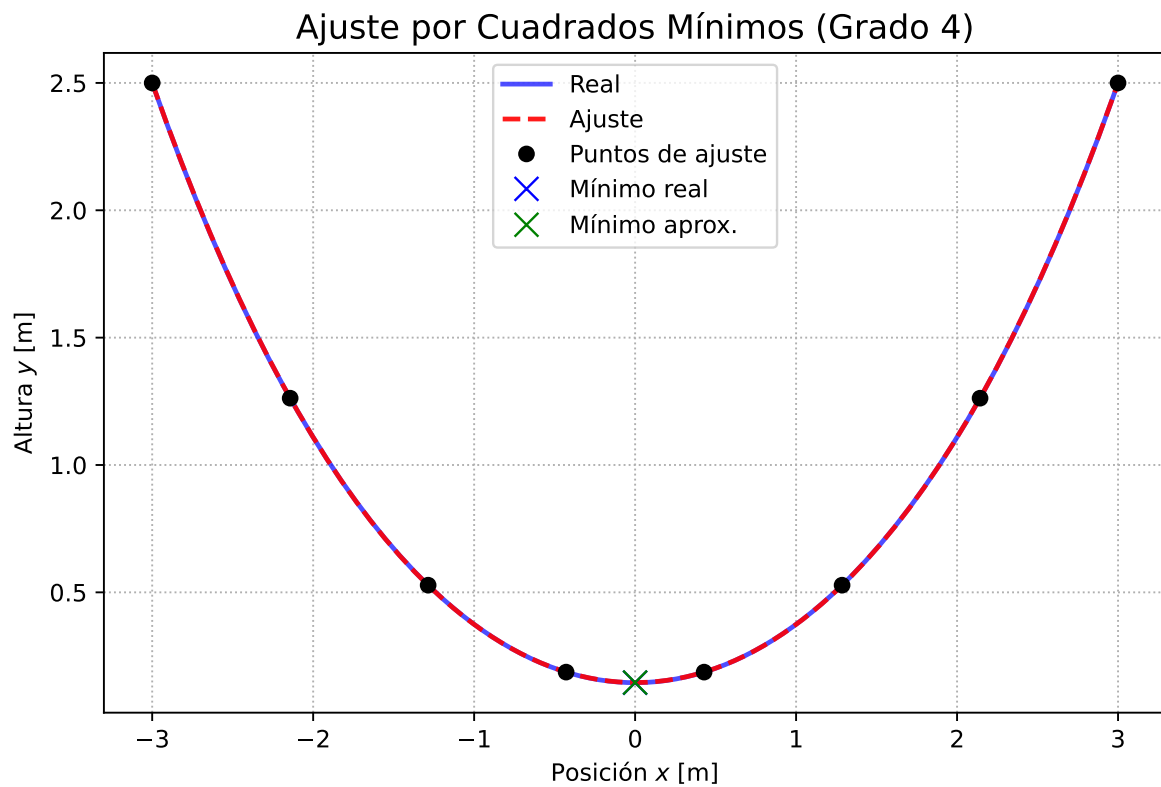
c0 = 0.145821
c1 = -0.000000
c2 = 0.224066
c3 = 0.000000
c4 = 0.004167

```

Altura mínima real de la sogá: 0.145328 m

Altura mínima aproximada $P_4(0)$: 0.145821 m

Error absoluto en la aproximación: 4.922001×10^{-4} m



(d) Separación máxima

Note

¿A partir de qué distancia de separación entre postes la sogla comienza a tocar el suelo?
Calcule la distancia con un error menor a $1mm$.

Para resolver a qué distancia la sogla toca el suelo, tenemos dos ecuaciones que deben cumplirse simultáneamente en el estado límite:

Longitud de la sogla:

$$\frac{2}{\gamma} \sinh(\gamma a) = L \rightarrow \sinh(\gamma a) = \frac{L}{2} \gamma$$

Altura mínima nula:

$$\frac{1}{\gamma} (1 - \cosh(\gamma a)) + h = 0 \rightarrow \cosh(\gamma a) = 1 + h\gamma$$

Elevando al cuadrado ambas ecuaciones y sumando:

$$\sinh^2(\gamma a) + \cosh^2(\gamma a) = (1 + h\gamma)^2 + \left(\frac{L}{2}\gamma\right)^2$$

Nuevamente, por la identidad fundamental de las funciones hiperbólicas:

$$1 = (1 + h\gamma)^2 + \left(\frac{L}{2}\gamma\right)^2$$

$$1 + 2h\gamma + h^2\gamma^2 - \frac{L^2}{4}\gamma^2 = 1$$

$$2h + \left(h^2 - \frac{L^2}{4}\right)\gamma = 0$$

$$\gamma_{limite} = \frac{2h}{\frac{L^2}{4} - h^2}$$

```
gamma_limite = (2 * h) / ((L**2 / 4) - h**2)
print(f"Valor límite de gamma: {gamma_limite:.8f}")
```

Valor límite de gamma: 0.51282051

Ahora resta encontrar el valor de a que corresponde a este γ_{limite} utilizando la ecuación de la altura mínima y buscamos la raíz de la siguiente función:

$$f(a) = \cosh(\gamma_{limite}a) - (1 + h\gamma_{limite}) = 0$$

Como debemos encontrar esta solución con un error mínimo a $1mm$, y teniendo en cuenta nuevamente la simétrica con respecto al $x = 0$, el margen de error para a es de $0,001/2 = 0,0005m$.

Podemos usar el método de la bisección para encontrar el valor de a que satisface esta ecuación con la tolerancia requerida. Para esto, es necesario elegir un intervalo inicial que contenga la raíz (es esperable que esté en el intervalo $[1, 0; 3, 0]$), y luego iterar hasta que el error máximo del intervalo sea menor a $0,0005m$.

```
def f_a(a):
    return np.cosh(gamma_limite * a) - (1 + h * gamma_limite)

# Implementación del Método de Bisección
def biseccion(func, lim_inf, lim_sup, tol_error):
    a_n = lim_inf
    b_n = lim_sup
    n = 0

    print(f"{'Iter':<5} | {'Extremo Izq':<12} | {'Extremo Der':<12} | {'Medio (a_aprox)':<15.6f} | {'Error':<15.6f}")
    print("-" * 68)

    # El ciclo continúa mientras el error máximo del intervalo sea mayor a la tolerancia
    while (b_n - a_n) / 2.0 > tol_error:
        c_n = (a_n + b_n) / 2.0
        error_actual = (b_n - a_n) / 2.0
        print(f"{'n':<5} | {'a_n':<12.6f} | {'b_n':<12.6f} | {'c_n':<15.6f} | {'error_actual':<15.6f}")

        # Evaluamos el punto medio
        if func(c_n) == 0:
            return c_n # Encontramos la raíz exacta de casualidad
        elif func(lim_inf) * func(c_n) < 0:
            b_n = c_n # La raíz está en la mitad izquierda
        else:
            a_n = c_n # La raíz está en la mitad derecha

    n += 1
```

```

# Calculamos el valor final que cumple la tolerancia
c_final = (a_n + b_n) / 2.0
error_final = (b_n - a_n) / 2.0
print(f"{n:<5} | {a_n:<12.6f} | {b_n:<12.6f} | {c_final:<15.6f} | {error_final:<15.6f}")

return c_final

tolerancia = 0.0005

a_calculado = biseccion(f_a, 1.0, 3.0, tolerancia)
distancia_total = 2 * a_calculado

print("-" * 68)
print(f"Distancia del centro al poste (a): {a_calculado:.4f} m")
print(f"Separación total entre postes (2a): {distancia_total:.4f} m")

```

Iter	Extremo Izq	Extremo Der	Medio (a_aprox)	Error Maximo
0	1.000000	3.000000	2.000000	1.000000
1	2.000000	3.000000	2.500000	0.500000
2	2.500000	3.000000	2.750000	0.250000
3	2.750000	3.000000	2.875000	0.125000
4	2.750000	2.875000	2.812500	0.062500
5	2.812500	2.875000	2.843750	0.031250
6	2.843750	2.875000	2.859375	0.015625
7	2.843750	2.859375	2.851562	0.007812
8	2.851562	2.859375	2.855469	0.003906
9	2.855469	2.859375	2.857422	0.001953
10	2.857422	2.859375	2.858398	0.000977
11	2.858398	2.859375	2.858887	0.000488

Distancia del centro al poste (a): 2.8589 m
Separación total entre postes (2a): 5.7178 m

Finalmente, la distancia entre postes donde la sog a comienza a tocar el suelo es $2a = 5,7178m$.